AutonomousRiderJS: High-Performance Multi-Agent Coordination, Evolutionary Partitioning, and Hybrid Planning in Deliveroo.js

Leonardo Collizzolli - 248810
Email: leonardo.collizzolli@studenti.unitn.it

Edoardo Fiorentino - 248444
Email: edoardo.fiorentino-1@studenti.unitn.it

Abstract*—This report presents `AutonomousRiderJS`, a highly optimized deliberative multi-agent system designed for the Deliveroo.js benchmark environment. Leveraging Belief-Desire-Intention (BDI) architecture, we detail our concrete contributions toward solving the performance, herding, coverage, and deadlocking problems inherent in real-time multi-agent grid environments. Our architecture combines (1) an optimized, concurrency-hardened BDI control loop with temporal parcel decay and custom exception filters; (2) a lazy Breadth-First Search (BFS) distance caching system reducing pathfinding overhead by three orders of magnitude; (3) a two-level online Evolutionary Algorithm (EA) that dynamically partitions map regions based on opponent pressure and solves the Travelling Salesperson Problem (TSP) for patrol routes, giving map coverage while idle; (4) a stateful communication protocol supporting claims with preemption and opportunistic/positional relay handoffs (including dead-end retreat choreography); and (5) a PDDL planner integrated directly into the BDI plan library with a circuit-breaker failover mechanism. A paired benchmark campaign over all 19 official validation maps (1280 agent-runs) quantifies the resulting trade-offs, and in particular shows that the cost of PDDL planning in this setting is an almost problem-size-independent 1.6 s solver round trip, whose damage is governed not by the size of the planning problem but by how fast parcels decay and how long the delivery cycle it is inserted into already takes.

I. Introduction and Architecture Overview

The Deliveroo.js benchmark environment presents a dynamic grid-world where autonomous agents must gather and deliver packages while competing against adversaries. In this setting, the central challenge is to coordinate a team of two agents in real-time, under high volatility, tight step latency (e.g., 50 ms movements), and local sensing constraints.

Rather than detailing standard BDI definitions, we focus directly on our main contribution: the architectural changes that transform a standard reactive agent into a high-performance, cooperative deliberative system.

Crucially, the system is designed to work in two execution modes: (1) **Solo Mode** (single-agent, where the agent behaves as a fully independent BDI rider to maximize points on single-agent maps), and (2) **Team Mode** (multi-agent, where a pair of teammates coordinate dynamically). If team mode is disabled (the default setting, i.e. when neither the environment variable `TEAM` nor the CLI argument `-team` is set to 1), all multi-agent mechanisms become a clean no-op, guaranteeing that single-agent behavior remains completely unchanged. When team mode is active, the coordination layer initiates communication and division of labor. The repository is organized into distinct functional units:

- `src/autonomousRider.js`: Main entry point, managing the initial handshake, the sensing hooks that trigger belief revision and options generation, and the start-up of the intention loop (`agent/Agent.js`) and of the coordination layer.
- `src/config/`: `config.js` (host, identity, and the `team`/`pddl` switches, settable either by environment variable or by cross-platform CLI argument).
- `src/agent/beliefs/`: The belief base. Contains `beliefs.js` (shared state, including team state), `parcels.js` (belief revision and temporal decay), `me.js` (self tracking), `otherAgents.js` (competitor and teammate tracking), and `map.js` (graph structure, delivery spots, and spawners).
- `src/agent/intention/`: Intention definitions, encapsulating execution wrappers, stop codes, and dynamic validity rules (`isStillValid()`).
- `src/agent/intentionRevision/`: The 2-slot queue scheduler implementing swap hysteresis, failure tracking, and concurrency protections.
- `src/agent/reasoning/`: Option generation and scoring (`reasoning.js`) and the patrol-tour optimizer (`tourEA.js`).
- `src/agent/planning/`: The plan library (`PlanLibrary.js`, set of all possible plans), the PDDL interface (`PddlPlan.js`), along-path harvesting (`GoPickUp.js`), delivery (`GoDeliver.js`), pathfinding and giving-way (`BlindMove.js`), and the lazy BFS distance cache (`spawnerDistances.js`).
- `src/agent/coordination/`: Messaging protocol (`messages.js`), discovery, claim and handoff controllers (`coordination.js`), and the cooperative zone manager (`partitionEA.js`).
- `src/agent/benchmark/metrics.js`: Evaluation instrumentation, emitting machine-parsable `METRIC` records by intercepting the client and a few BDI hooks; a set of cheap counters during normal play.
- `benchmark/`: The headless testbench used in Section VI: `run-benchmark.js` (campaign runner) and `parse-logs.js` (CSV aggregation).

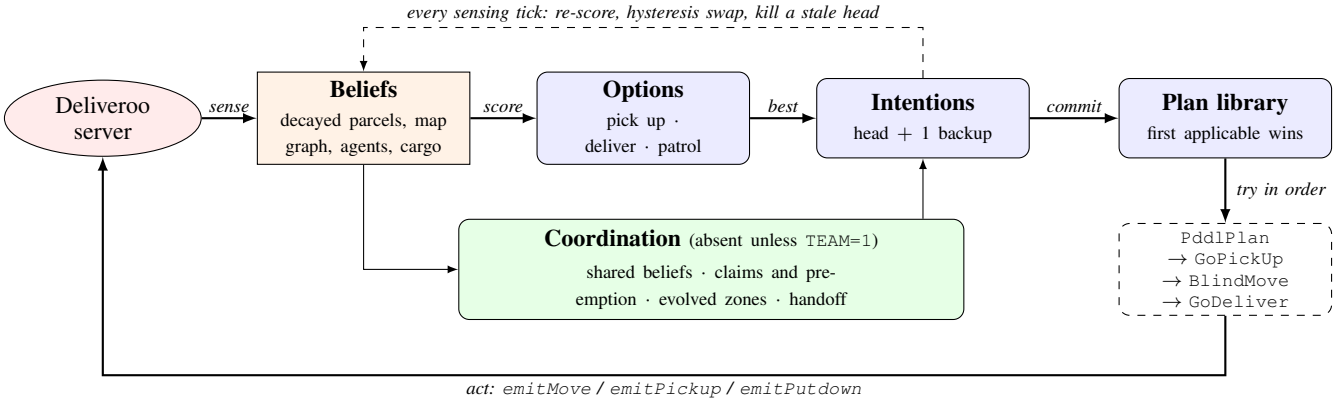

Fig. 1: The deliberation cycle. The left-to-right path is the easy half; the two feedback edges are the design. Every sensing tick re-scores the options and may swap the head of a two-slot queue or kill one whose target has vanished (dashed), which is what keeps a deliberative agent reactive in a world that moves during its own plans. Commitments are handed to an ordered plan library where `PddlPlan` is tried first and any failure simply falls through to the classical plans. The coordination layer reads the same beliefs and revises the same intentions, and is absent entirely in single-agent mode.

Figure 1 outlines the information flow. In subsequent sections, we detail how we solved critical bugs (such as the event-loop race enqueuing duplicate intentions), how we optimized pathfinding, how the team splits the map dynamically using an online Evolutionary Algorithm, and how PDDL is integrated as a first-class planning module.

II. Single-Agent Architecture: Deliberative Core & Performance Engineering

The individual agent is built around an optimized, concurrency-hardened deliberative loop. Sensed environment changes are parsed, merged into beliefs, scored, scheduled into a 2-slot intention queue, and executed.

A. Belief Revision and the Belief Base

The belief base is the central repository of our BDI agent, managing static and dynamic state across both single-agent (solo) and multi-agent (team) executions. The architecture is defined by three key belief sub-modules:

- `me.js` (Self-Tracking): Keeps track of the agent's identity, current score, coordinates (x, y), and cargo state (number of carried parcels, total carried reward, and carried parcel IDs). In both solo and team modes, the cargo state is not incremented when our own pickups and putdowns succeed: it is recomputed from scratch on every sensing step, by scanning the `carriedBy` field of the parcel snapshot the server sends us. Using the snapshot to update the stored information is idempotent and self-healing: every sensing event re-synchronizes the estimate with ground truth, so no error can accumulate, at the negligible cost of one pass over the currently perceived parcels.
- `otherAgents.js` (Opponent Tracking): Maintains a registry of other agents perceived in the field. When running in solo mode, all perceived agents are treated as opponents and factored into risk penalties (P_{risk}). When running in team mode, the teammate is identified during the initial HELLO handshake and is explicitly excluded from risk calculations, preventing the agent from treating its partner as a competitor.
- `parcels.js` (Temporal Parcel Decay): Since parcels decay continuously on the server, the agent maintains a local database (`storedParcels`) containing the last perceived properties, a timestamp, and a visibility flag. Between physical sightings, the parcel's reward is continuously decayed locally using:

$$R(t) = R_{\text{sensed}} - \frac{\Delta t}{\text{PDI}}, \quad \text{where } \Delta t = \frac{t_{\text{now}} - t_{\text{sensed}}}{1000} \tag{1}$$

Here, t and t_{sensed} are millisecond timestamps, Δt is the elapsed time in seconds, and PDI is the Parcel Decaying Interval. An early bug involving an unnecessary `parseInt()` round on the decay term truncated decimal decrements, causing decay to evaluate to 0 and preserving dead parcels indefinitely. Replacing it with floating-point decay solved this.

Furthermore, a rare server-side race condition (such as sensing a parcel precisely during its delivery or putdown) can return undefined coordinates or NaN rewards. Ingesting these poisoned entries broke the options-generation loop, causing it to crash with several errors/second and completely locking the agent. To harden the belief base, we implemented an ingestion guard at the single entry point through which sensed parcels enter beliefs:

```
if (!Number.isFinite(p.x) || !Number.isFinite(p.y)
    || !Number.isFinite(p.reward)) continue;
```

and made parcel deletion NaN-safe by replacing standard inequalities with a strict negative negation: `!(parcel_current_reward > 0)`.

B. Decision Model and Intention Revision

Options are generated on every sensing event. The decision model ranks options using a comprehensive scoring

function `calculateScore`. Because points are awarded only upon delivery, a pickup intention targeting tile T is evaluated by computing its expected net reward at the moment of delivery. Evaluating the full round-trip trajectory ($d(\text{me}, T) + d(T, \text{nearest\_delivery})$) *a priori* prevents the agent from falling into "greedy traps", such as pursuing high-value parcels located in isolated map corners where transit decay would reduce final payout to zero, while ensuring that adding new cargo does not cause existing carried items to decay past their net worth during the extended journey. A single `emitPickup()` collects *every* parcel on the agent's tile: we sum the rewards of all n_T parcels lying on T and charge the post-pickup decay for all of them. Writing $R_T = \sum_{i \in T} R_i$ and c for the number of parcels already carried,

$$R_{\text{pickup}} = (R_T - n_T \cdot \text{PDR} \cdot d(\text{me}, T)) + (R_{\text{carried}} - c \cdot \text{PDR} \cdot d(\text{me}, T)) \quad (2)$$

$$R_{\text{delivery}} = R_{\text{pickup}} - (c + n_T) \cdot \text{PDR} \cdot d(T, \text{nearest\_delivery}) \quad (3)$$

$$\text{Score} = \gamma_{\text{zone}} \cdot \gamma_{\text{mate}} \cdot R_{\text{delivery}} - P_{\text{risk}} - P_{\text{failure}} \quad (4)$$

where $d(a, b)$ is the graph distance. The team terms are reward multipliers (both set to 1 in Solo Mode):

- $\gamma_{\text{zone}} = 0.65$: applied if the parcel lies outside our evolved partition zone (Section IV).
- $\gamma_{\text{mate}} = 0.50$: applied when the teammate is at least twice as close to the target tile ($d(\text{mate}, T) < \frac{1}{2} d(\text{me}, T)$).

Crucially, these multipliers are applied only when R_{delivery} is greater than zero; scaling negative rewards would invert incentives by making discouraged options appear better.

The risk penalty P_{risk} is defined as

$$P_{\text{risk}} = 10 \cdot |\{o \in \text{Opponents} \mid d(o, T) < d(\text{me}, T)\}|, \quad (5)$$

which subtracts 10 points for each competitor closer to tile T than the agent. To prevent false penalties, this term is enforced only for local targets within a 5-tile radius ($d(\text{me}, T) \leq 5$), focusing deliberation on immediate races rather than long-range predictions, and considers exclusively fresh opponent sightings ($\text{age}(o) \leq 5$ s) to filter out obsolete belief data.

The failure penalty $P_{\text{failure}} = M_{\text{fail}} \cdot f(k)$ penalizes intentions that have recently failed, where $f(k)$ is the number of recent failures for intention k. To forgive temporary obstructions, $f(k)$ decays by 1 failure every 10 seconds. The penalty multiplier M_{fail} uses a fixed 10-point penalty for pickups to quickly pivot away from contested parcels, whereas for deliveries it scales proportionally as $\frac{1}{3} R_{\text{delivery}}$ to reflect the variable stakes of cashing in already-carried cargo. Furthermore, if an intention fails 5 times, it is temporarily blacklisted for 10 seconds.

To translate options into action, the agent utilizes a 2-slot intention queue (`intention_queue`), consisting of an active head intention and a single backup intention. We deliberately restricted the queue length to 2 slots to optimize the agent's reactivity to high environmental dynamics. In the Deliveroo.js environment (where new parcels spawn, decay, or get claimed, and competitors move constantly), enqueuing a long chain of actions makes the agent sluggish and unresponsive, causing it to waste steps executing outdated commitments. Conversely, a 1-slot queue would leave the agent idle at the end of an action while option generation and planning execute. The 2-slot queue provides a balanced trade-off: it ensures action continuity via the backup slot (enabling pre-planning and zero-delay execution transition) while keeping the agent highly reactive to new, higher-reward opportunities, which can easily swap the backup slot or preempt the head if they exceed the hysteresis margins. The revision mechanism incorporates three key improvements:

1) **Hysteresis Margin.** If the challenger option is comparable to the currently executing head, the agent could oscillate, turning back and forth between two tiles without making progress. To prevent this, we enforce a hysteresis threshold:

$$\text{Score}_{\text{challenger}} > \text{Score}_{\text{incumbent}} \times 1.10 \quad (6)$$

which is applied only when the incumbent score is positive to avoid inverting negative inequalities.

2) **Concurrency Hardening.** The `push` and `swap` methods were originally asynchronous. Because of JavaScript's event-loop scheduling, multiple concurrent sensing events (e.g., parcel updates and agent updates firing back-to-back) could interleave their checks, enqueuing duplicate copies of the same intention (e.g., `[p35, p35]`). We eliminated this race condition by making all queue-manipulation and comparison steps strictly synchronous, executing stops and swaps synchronously and delegating async waits to background tasks.

3) **Ghost-Head Killing.** We enforce an active validity check (`isStillValid()`) inside the executing loop. If a parcel disappears mid-walk, the head intention is immediately killed. Crucially, any projected `go_deliver` intention queued behind an aborted pickup is also purged, preventing the agent from walking to a delivery zone empty-handed.

C. Movement: Predictive Avoidance and Asymmetric Replanning

Executing a `go_to` is where the agent meets the other bodies on the map, and two design choices in `BlindMove` matter more than the pathfinding itself.

Predictive collision avoidance. A tile is treated as occupied not only when an agent stands on it, but also when an agent is about to step into it. Each sighting in `otherAgents.js` is compared against the previous one to infer a movement direction, and `isTileFree` rejects the tile immediately ahead of a moving agent:

```
if (agent.direction === 'right'
    && agent.x + 1 === next[0]
    && agent.y === next[1]) return true; // occupied
```

This one-step opponent model is deliberately crude, it assumes each agent continues straight, but it is enough to stop the

most common collision, walking into the tile someone else is entering. Since every rejected move costs a server penalty, avoiding the collision is worth more than the occasional unnecessary detour. Sightings older than two movement steps are ignored, so a stale position never blocks a corridor.

Optimistic first path, pessimistic replans. The initial path is computed over the *full* graph, temporarily blocked tiles included; every replan is computed over the graph with those tiles removed:

```
// initial: allowTemporaryBlocked = true
path = await findBestPath(me, target, true);
// replan: blocked tiles dropped from the graph copy
path = await findBestPath(me, target);
```

The asymmetry is intentional. Most blockages are transient (an agent passing through), so committing to the shortest route and waiting a step usually wins; a tile that has actually blocked us is evidence, and only then is it worth paying for a detour. A blocked tile ages out after three movement durations, so the pessimism is short-lived. The replan budget of three attempts is reset on every successful move, making it a limit on consecutive obstruction rather than on the journey: an agent crossing a busy map may replan many times, while one truly walled in fails fast instead of grinding.

D. Performance Engineering: Lazy BFS Distance Cache

Initially, calculating distances for the scoring function relied on running Dijkstra's algorithm. Profiling the code showed that a single Dijkstra call took 0.233 ms. During options generation, the scoring function performs up to 500 distance queries per tick (evaluating all parcels, delivery spots, teammate positions, and opponents), resulting in $\sim$100 ms of event-loop blockage. This delay caused movement lag, making agents miss movement windows.

We resolved this by implementing a lazy Breadth-First Search (BFS) cache (`spawnerDistances.js`). Since the map grid is static and all edges have a cost of 1, BFS is sufficient. The cache generates distance tables from a target tile T to all reachable nodes on demand. Once computed, the distance between any node N and T is retrieved in $O(1)$ time via a simple map lookup:

```
const getDistanceTable = (tileId) => {
    if (distFrom?.has(tileId)) return distFrom.get(
        tileId);
    let table = lazyTables.get(tileId);
    if (!table) {
        table = bfsFrom(graph, tileId);
        lazyTables.set(tileId, table);
    }
    return table;
};
```

III. Team Coordination and Cooperative Handoffs

When the `TEAM` flag is active, coordination is handled by two independent agent processes communicating over the Deliveroo message bus. Crucially, the messages are signed using a shared `TEAM_SECRET` to distinguish teammates from opponents.

A. Communication Protocol and Heartbeat Liveness

The protocol supports presence discovery, belief sharing, and mental-state coordination, and exercises all three messaging primitives offered by the platform (shout, say, and ask). Every message is wrapped in an envelope carrying the shared secret and a timestamp, as summarized in Table I, since shouts reach every agent on the server:

TABLE I: Coordination Protocol Messages

Message	Primitive	Description/Payload
`HELLO`	Shout	Discovery announcement (every 2 s)
`HELLO_ACK`	Say	Directed reply, so both sides learn the ids
`PARCELS`	Say	Directly visible free parcels + status heartbeat (position, cargo count and reward)
`AGENTS`	Say	Sensed opponents (own observations only)
`CLAIM`	Say	Intention key, target and local score, sent when execution starts (TTL 30 s)
`RELEASE`	Say	Relinquish target claim
`HANDOFF`	Ask	Handoff proposal (carried reward and count, proposer's best local score, position, boxed flag)
`HANDOFF_DONE`	Say	Drop notification (parcel IDs, coordinates)
`ZONES`	Say	Partition assignment and version (leader broadcast)

A critical liveness issue occurred on maps where agents stood still (e.g., waiting for spawns). Because sensing events on the server only trigger on movement, stationary agents did not receive sensing updates. Consequently, their coordination data became stale, which disabled handoffs. To resolve this, we modified the `PARCELS` message to act as a heartbeat: it is sent periodically even when the local parcel list is empty, sharing the agent's current position and cargo state. If a teammate goes silent for more than 10 seconds, the active partition is dropped, and the remaining agent reverts to searching the entire map.

B. Division of Labor and Anti-Herding

If both agents receive identical belief updates, they will calculate the same best options and compete for the same parcel. We introduced three coordination mechanisms to prevent herding:

- **Soft Zone Preference (γ_{zone}).** We apply a multiplier of $\gamma_{\text{zone}} = 0.65$ to the score of parcels lying outside the agent's assigned zone (described in Section IV).
- **Teammate Proximity Discount (γ_{mate}).** If the teammate's graph distance to a parcel is less than half of ours ($d(\text{mate}, T) < \frac{1}{2} d(\text{me}, T)$), we apply a multiplier of $\gamma_{\text{mate}} = 0.50$ to our score. This soft discount allows the closer agent to collect the parcel while still letting the farther agent step in if no other options are available.
- **Under-Teammate Skip.** If a parcel is located directly on the teammate's tile, it is skipped during option generation, as the teammate will collect it automatically.

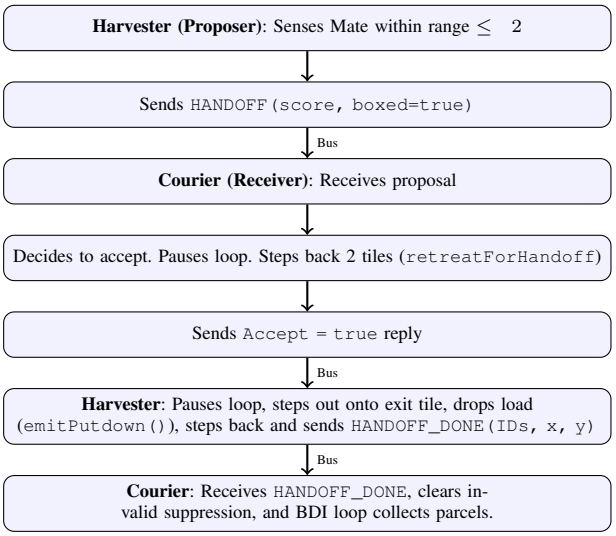

Fig. 2: Handoff choreography sequence for the corridor deadlock scenario.

C. Retroactive Preemption and Handoff Choreography

When an agent starts executing an intention, it broadcasts a `CLAIM` message containing its score. If the receiver has the same intention queued or running, it compares the scores. If the teammate's score is higher, the receiver yields and stops its intention:

```
if (isClaimedByTeammate(intentionKey, myScore)) {
    intention.stop(QUEUE_SWAP_STOP_CODE);
}
```

This retroactive preemption prevents agents from racing each other when option updates interleave. Score ties are broken by comparing agent ids, so both sides reach the same verdict without another round trip.

Randomized Symmetry Breaking. When two teammates carrying cargo meet, symmetric belief updates cause both agents to evaluate handoff conditions and transmit a `HANDOFF` proposal simultaneously. Because an outgoing proposal activates a local lock (`handoffInFlight`), simultaneous cross-proposals are mutually rejected, wasting the encounter. Rather than implementing a heavy multi-round negotiation protocol, we introduce probabilistic symmetry breaking: each agent refrains from proposing 50% of the time:

```
    if (!blockedByMate && Math.random() < 0.5)
        return;
```

This simple 50% coin-flip desynchronizes proposal timing, guaranteeing that one agent proposes while the other is ready to accept. The jitter is bypassed when agents physically block each other (`blockedByMate`), prioritizing immediate corridor deadlock resolution over desynchronization delay.

The handoff mechanism allows one agent (the harvester) to transfer its cargo to the other (the courier). The receiver accepts a handoff if it meets either of two rules:

- **Economic Rule.** The proposer has better local harvesting options ($\text{Score}_{\text{proposer\_best}} > \text{Score}_{\text{receiver\_best}} \times 1.15$). This is protected by a 15-second cooldown to prevent cargo ping-ponging.
- **Positional Relay.** The receiver is closer to a delivery spot than the proposer, and its local harvesting options do not exceed the proposer's. This uses a shorter 5-second cooldown.

Corridor Deadlock Resolution Protocol. Handoff execution requires coordinated steps to avoid collisions and lost parcels. If the proposer is "boxed in" (e.g., standing in a dead-end corridor with the receiver blocking the only exit), they execute the retreat choreography shown in Figure 2. The receiver backs up 2 tiles, the proposer steps out, drops the parcels, and steps back in, allowing the receiver to collect the cargo. To prevent the proposer from immediately picking its own dropped parcels back up, we temporarily suppress the dropped parcel IDs using the `invalidOptions` map.

IV. Evolutionary Strategies: Zoning and Tour Optimization

Rather than using pre-computed spatial zones, our system runs two evolutionary algorithms online during the match. These algorithms use model-based fitness functions evaluated against live observations.

A. Level 1: Dynamic Map Partitioning EA

To partition map spawners between teammates without hardcoded regions, the leader agent periodically runs an online evolutionary algorithm (`partitionEA.js`) every 2 seconds in short 15 generation bursts. The algorithm assigns each spawner tile to an agent using binary bitstring genomes initialized with spatial medians.

The multi-objective fitness function minimizes a weighted combination of workload balance, spatial compactness, and switching distance:

$$\text{Fitness} = w_b \cdot \text{Cost}_{\text{balance}} + w_c \cdot \text{Cost}_{\text{compactness}} + w_s \cdot \text{Cost}_{\text{switch}} \tag{7}$$

Workload balance is computed from the dynamic spawner weight W_{spawner}, which discounts spawners under heavy opponent pressure while boosting active parcels:

$$W_{\text{spawner}} = \frac{1}{1 + \text{OpponentPressure}} + 0.5 \cdot \frac{R_{\text{uncollected}}}{R_{\text{max\_possible}}} \tag{8}$$

Compactness penalizes non-adjacent spawner clusters via k-nearest neighbor graphs ($k = 4$), while switching distance measures transit cost to assigned zones. A 10% hysteresis margin prevents boundary flickering before broadcasting updates to the follower.

B. Level 2: Patrol Tour EA (TSP)

When an agent is idle, it wanders to search for parcels. Rather than walking to the farthest spawner, each agent runs an online Travelling Salesperson Problem (TSP) solver (`tourEA.js`) to find a short, cyclic patrol tour. In solo mode, the tour covers all spawners on the map. In team mode, it covers only the spawners belonging to the agent's assigned

zone partition, preventing agents from wandering into each other's territory.

- **Genome**: A permutation of the spawner IDs in the agent's active zone (the assigned zone in team mode, or all spawners on the map in solo mode).
- **Fitness**: The total cycle length in graph distance. The patrol set is restricted to the agent's connected component to avoid impossible paths.
- **Operators**: Order Crossover (OX) and 2-opt segment-reversal mutations, running in 10-generation bursts.

V. PDDL Integration and Fallback Planning

We integrated automated planning by placing a PDDL planner first in our plan library. If PDDL is enabled, the agent attempts to solve pickup and delivery tasks using a STRIPS domain with negative preconditions.

A. Domain Definition and Problem Generation

The domain defines predicates tracking agent positions, parcel locations, carried cargo, connectivity, and blockages:

```
(define (domain deliveroo)
  (:requirements :strips :negative-preconditions)
  (:predicates
    (is-agent ?a) (is-tile ?t) (is-parcel ?p)
    (at-agent ?a ?t) (at-parcel ?p ?t) (carrying ?a
        ?p)
    (connected ?t1 ?t2) (blocked ?t)
    (is-delivery-zone ?t) (delivered ?p)
  )
  (:action move
    :parameters (?a ?from ?to)
    :precondition (and (is-agent ?a) (is-tile ?from)
         (is-tile ?to) (at-agent ?a ?from) (
        connected ?from ?to) (not (blocked ?to)))
    :effect (and (at-agent ?a ?to) (not (at-agent ?a
         ?from)))
  )
  ...
)
```

1) What the solver is actually asked: The problem instance is rebuilt for every intention, and its shape explains both the planner's strength and its cost.

Objects and static facts. Every walkable tile becomes an object, every adjacency a `connected` fact, and every delivery tile an `is-delivery-zone` fact. This part depends only on the map, which never changes, so it is compiled once at startup and reused verbatim; only the dynamic half is rebuilt per call.

Dynamic facts. Our position (`at-agent me`), the parcels we carry (`carrying me ?p`), the free parcels relevant to this intention (`at-parcel ?p ?t`), and the tiles to keep clear (`blocked ?t`), the latter drawn from the temporary-blocked list plus every opponent sighted in the last second. Opponents are thus modelled as *static obstacles* frozen at their last known position: the domain has no notion of time, so a plan is optimal against a snapshot of a world that keeps moving while the solver thinks.

Goals: Always delivery, never intermediate pickup. This design choice fundamentally shapes the solver's behavior. Rather than setting intermediate goals such as reaching a tile or acquiring a parcel, every PDDL goal is formulated as a conjunction of terminal delivery predicates `(delivered ?p)`. For any triggering intention toward a target parcel T, the goal includes T, all items currently in cargo, and any free parcel p satisfying a topological detour constraint ≤ 2 (i.e., $d(\text{me}, p) + d(p, T) \leq d(\text{me}, T) + 2$). Consequently, a pickup intention does not merely request a path to a target tile; it asks the solver to find a complete harvest-and-deliver sequence that collects en-route parcels and deposits all cargo into a delivery zone.

The narrower formulation was available and was rejected deliberately. Asking for `(carrying me ?p)` on the target parcel alone would have matched the granularity of the BDI layer exactly: one intention, one plan, one commitment, with the delivery re-scored separately the way the classical pipeline does it. It would also have made the planner redundant. Reaching a parcel on a uniform-cost grid is a shortest-path query, and our classical plans already answer it with a single Dijkstra run over the cached map graph in a fraction of a millisecond; paying a solver round trip for the same answer would buy a slower version of something we had. The conjunction of `(delivered ?p)` goals is the smallest formulation for which the solver decides something the hand-written plans genuinely cannot, namely the *order* in which several parcels and a delivery zone are visited. That is a combinatorial choice rather than a path, and it is the only reason a planner earns its place in the library at all.

The price is a coarser granularity of commitment. One PDDL plan absorbs a whole pickup-to-delivery cycle into a *single* intention, where the classical pipeline splits it into a `go_pick_up` followed by a separately-scored `go_deliver`. The intention-revision machinery of Section II operates between intentions, so during a PDDL plan the delivery half is never re-scored on its own, never announced with a `CLAIM`, and never subject to the delivery-spot arbitration of Section III. Revision can still preempt the plan wholesale when a better option appears, but that is an all-or-nothing choice where the classical pipeline would simply re-decide at the delivery boundary. On a stable map the longer commitment is a strength; on a volatile one it compounds the staleness measured in Section VI.

This granularity mismatch triggered our sharpest integration bug. In the classical architecture, a `go_pick_up` validity check (`isStillValid()`) verifies that the target parcel remains uncollected on the ground in `storedParcels`. Once picked up, the classical plan completes immediately, delegating delivery to a subsequent intention. In contrast, a unified `PddlPlan` executes both harvesting and delivery within a single long-running intention. When the agent successfully collected the target, belief revision instantly transferred the parcel from `storedParcels` into the agent's cargo. The active validity loop interpreted its absence from the ground as a lost target, aborting the `PddlPlan` immediately after pickup, precisely when the delivery leg was set to execute. Resolving this required updating the validity guard to recognize that a target residing in the agent's own cargo represents valid progress rather than plan failure.

Cooperative donation. If a teammate is within graph distance 3, they enter the PDDL problem as agent `mate`. Because our agent can only execute its own actions, any plan steps assigned to `mate` are skipped, causing postcondition checks to fail and falling back to classical planning. The main multi-agent pattern we execute is an intentional cargo drop (`putdown`) next to the teammate. The executor turns this step into a handoff by dropping the parcels and sending a `HANDOFF_DONE` message for the teammate to collect them.

B. Circuit Breaker and Fallback Architecture

Because the planner is a separate service driven over HTTP (submit a job, then poll until it finishes) solver latency is high and largely irreducible even with the service running locally: we measure a mean round trip of about 1.6 s (Section VI), which is problematic in a game with 50-200 ms movement steps. We designed a fallback architecture to mitigate it:

1) PDDL planning is attempted first. If the solver fails, times out, or returns an empty plan, the execution falls back to classical plans (e.g., BFS-Dijkstra pathfinding).
2) **Circuit Breaker.** If the solver fails 3 times consecutively, the PDDL planner is disabled for the rest of the session to keep the agent from standing still.

VI. Experimental Validation and Results

A. Methodology

Measurements come from a headless testbench (`benchmark/`) that starts a fresh local server per map, connects the agents, and lets every run play for three minutes. Instrumentation is non-invasive: `metrics.js` intercepts the API client and a handful of BDI hooks and emits one machine-parsable `METRIC` record per event, so no game logic is modified in order to be measured. `parse-logs.js` aggregates the logs into CSV tables and `report-tables.js` regenerates Tables II–IV from them.

The campaign is a *paired* design: the two configurations play *simultaneously on the same server*, so they see the same parcel spawns, the same congestion and the same random seed. Single-agent maps get six independent agents, half with PDDL; coordination maps get four teams of two, half with PDDL, each team with its own `TEAM_SECRET` so teams never cross-link over the broadcast bus. Every map is played ten times, for a total of 1280 agent-runs across all nineteen validation maps, against a solver that stayed reachable throughout.

Three caveats. Because both configurations draw on one shared parcel supply, the numbers measure *competitive* rather than absolute performance: the slower one also loses parcels to the faster one, so the gap is wider than two isolated runs would show. Spawn randomness dominates any single run, so we read trends over ten runs rather than point estimates. And `25c2_hallway` is the one map where the corridor fits a single team, so there the configurations alternate across runs instead of sharing a server; with zero reward variance and a fixed spawn interval that comparison is still sound, but it is across runs rather than within them.

B. Map Taxonomy

Averaging over all maps hides the mechanism, because the validation set spans very different environments. We therefore classify each map by parameters read from the level files (Table II) and compare the two configurations within each class (Table III):

- **Planning-problem size**: the number of walkable tiles, from a twenty-tile corridor to a nine-hundred-tile open map. This *is* the size of the PDDL problem we generate, since every tile becomes an object and every adjacency a `connected` fact.
- **Contention**: single-agent maps, where six bodies compete for one parcel supply, against coordination maps, where eight do.

C. The Cost of Planning is Latency, not Problem Size

A solver round trip averages about 1.6 s, and it barely moves as the problem grows: over the campaign's forty-five-fold range in tile count, mean solve time rises by roughly a tenth. The overhead is a fixed charge for talking to an HTTP service, not the price of grounding or search. The planner is not slow because our problems are large; it is slow because it is remote. Measured against a movement step of 50-200 ms, one round trip is eight to thirty moves spent standing still.

That charge is paid several times per delivered parcel (once per intention, and again after every fallback) which stretches the whole delivery cycle by a factor of three and leaves the PDDL configuration with roughly a third of the baseline score. Cargo also waits longer in hand before it is banked, and on decaying maps that wait is subtracted directly from the reward.

The slowdown accounts for the entire loss, which is worth confirming because a planner could equally have lost points by choosing badly. Across maps, how far a configuration's delivery cycle stretches predicts its score almost exactly ($r = -0.88$). The plans the solver returns are not worse; they simply arrive too late, and too few of them arrive at all.

D. What Determines Where Planning Survives

If the cost is a fixed delay per intention, then what decides whether it is affordable is what the delay is inserted into. A fixed charge disappears into a long delivery cycle and dominates a short one, so maps where the baseline is itself unhurried tolerate the planner far better than quick ones.

The extremes make the point without any grouping. On `25c2_9`, where sixty delivery tiles put a drop-off almost everywhere and the baseline still needs the better part of half a minute per parcel, PDDL keeps about half the baseline score, reaching the best result in the campaign. On `25c1_9`, where the baseline turns a delivery around in four seconds, it keeps less than a tenth: the same 1.6 s is a rounding error in the first case and several times the entire cycle in the second. The planner does not need a *simple* world, it needs a *slow* one. The size trend in Table III is the same effect seen from another angle, since a bigger map means longer walks between parcels, and a longer walk is exactly what a fixed delay disappears into.

TABLE II: Validation maps, their classification, and the paired PDDL/baseline result (mean over runs; pts/min per agent).

		Map				Score (pts/min)			Solver	
Map	Runs	Tiles	Spawn.	Deliv.	Parcels	Baseline	PDDL	Ret.	ms	Empty
25c1_1	10	900	895	5	20	117.1	38.1	33%	1780	22%
25c1_2	10	464	456	8	20	544.4	153.5	28%	1572	19%
25c1_3	10	332	16	12	10	224.7	49.6	22%	1513	14%
25c1_4	10	592	8	4	10	526.8	299.6	57%	1580	12%
25c1_5	10	432	289	7	10	583.2	310.4	53%	1502	11%
25c1_6	10	445	22	11	24	89.2	14.8	17%	1567	49%
25c1_7	10	177	10	2	10	222.3	56.4	25%	1493	19%
25c1_8	10	158	16	6	10	185.3	48.8	26%	1463	19%
25c1_9	10	252	14	9	10	526.6	40.4	8%	1484	33%
25c2_1	10	260	16	12	10	134.8	23.9	18%	1546	43%
25c2_2	10	552	32	4	10	114.0	30.4	27%	1708	35%
25c2_3	10	432	59	7	10	308.8	118.0	38%	1572	27%
25c2_4	10	445	25	25	24	269.3	50.1	19%	1744	43%
25c2_5	10	195	10	2	10	177.4	30.2	17%	1582	40%
25c2_6	10	236	16	4	16	222.9	63.4	28%	1568	53%
25c2_7	10	524	5	1	10	314.8	62.1	20%	1860	53%
25c2_8	10	526	16	7	5†	108.8	34.9	32%	1660	30%
25c2_9	10	465	30	60	5†	49.1	25.2	51%	1582	62%
25c2_hallway‡	10	20	1	1	10	39.1	16.7	43%	1474	45%

† no level file: played with the backend default settings. ‡ only one team fits, so the two configurations alternate across runs instead of sharing a server.

TABLE III: PDDL vs. baseline per class of map. Retention is the PDDL score as a fraction of the baseline score.

Class	Maps	Base pts/min	PDDL pts/min	Ret.	Solve ms	Empty plans
Problem size (walkable tiles)						
Small (<300)	7	226.5	42.0	19%	1526	38%
Medium (300-500)	7	284.7	98.3	35%	1586	33%
Large (>500)	5	226.8	84.6	37%	1722	35%
Contention						
Single agent (6 bodies)	9	335.5	112.4	34%	1550	23%
Teams (8 bodies)	10	184.8	47.8	26%	1642	43%

TABLE IV: Team-level results on the coordination maps: the two agents of a team summed, mean per team-run, baseline configuration over all runs. Only one team fits on 25c2_hallway, where the runner alternates configurations across runs, so that map contributes fewer team-runs.

Map	Teams	pts/min	Deliv.	Handoffs	Observed
25c2_1	20	269.7 ± 26	59.4	7.35	95%
25c2_2	20	228.0 ± 23	37.5	5.95	98%
25c2_3	20	617.6 ± 76	44.5	3.80	94%
25c2_4	20	538.7 ± 28	68.1	7.50	98%
25c2_5	20	354.7 ± 50	50.2	11.85	100%
25c2_6	20	445.8 ± 80	38.9	12.00	100%
25c2_7	20	629.7 ± 93	46.5	9.80	78%
25c2_8	20	217.6 ± 34	28.4	1.75	98%
25c2_9	20	98.3 ± 24	15.1	0.35	97%
25c2_hallway	5	78.3 ± 44	24.8	29.00	100%

Problem size predicts the solver's latency; travel time predicts whether that latency is affordable.

Contention is the second factor, and it acts on plan validity rather than on time. A call that comes back with an *empty plan* is one whose problem, generated a second and a half earlier, has since become unsolvable, almost always because the target parcel decayed away or was taken by somebody else. Adding two more bodies to a map nearly doubles that rate, and with it the number of solver calls wasted per parcel actually delivered.

One caution against reading too much into that rate. We first took it for the master variable, but across the full map set its correlation with retention is weak, precisely because of maps like 25c2_9 that are simultaneously very stale and very forgiving. Empty plans measure how out of date the planner's world model is by the time it answers; they do not, by themselves, measure what being out of date costs.

E. Team Coordination

Table IV reports the coordination maps at team granularity, summing the two agents of a team. Teams cover almost the whole walkable map within a run, which is the sweep the evolved patrol tour is designed to produce.

The handoff column is the more interesting one, because it varies by nearly two orders of magnitude across maps, and it varies in the direction the mechanism was designed for. On 25c2_9, whose sixty delivery tiles put a drop-off point almost everywhere, handoffs essentially never happen: nobody needs a courier. On 25c2_hallway, a single-file corridor with the spawner at one end, the delivery tile at the other, and no room for the agents to pass each other, the same code hands parcels over dozens of times per run. There the relay is not an optimization but the only way to score at all, and the agents settle into stable harvester and courier roles because the corridor fixes their order. Every other map falls between these two. That one unmodified policy spans both extremes, with no map-specific tuning, is the result we would point to for the coordination component.

F. Micro-benchmarks

Two component measurements complete the picture. The lazy BFS distance cache reduced a single distance query from

0.233 ms under Dijkstra to 0.2 μs, a thousandfold speedup that removes roughly 100 ms of event-loop blockage per options tick and keeps ticks under 2 ms. Movement reliability confirms the effect end to end: fewer than two per cent of issued moves are rejected by the server, so the agents are almost never fighting their own latency.

G. Limitations and Design Trade-offs

- **PDDL is a net cost in this environment, and the architecture is built to absorb that**. The measurements vindicate the plan-library design rather than the planner: because `PddlPlan` degrades to the classical plans on any failure, a high empty-plan rate costs latency but never correctness, and the circuit breaker had to disable planning in well under one per cent of runs. This is also why PDDL is opt-in. Its plans are genuinely richer, chaining several pickups and the delivery into one route, but richness does not pay for a second and a half of standstill at a 50 ms step. The honest conclusion is conditional rather than dismissive: the same planner behind an in-process solver, or in a slower and less contested world, would be a different proposition, and our results say precisely which properties of a map decide that.
- **No opportunistic walk-over collections**: agents do not collect parcels they merely walk over unless an intention says so. We chose this to preserve BDI purity, with `GoPickUp`'s along-path sub-intentions covering the useful cases.
- **Shared-server measurement**: as noted above, the paired design measures competitive rather than absolute performance, which amplifies the difference between the two configurations.

VII. Conclusion

By focusing on performance optimizations and coordination mechanisms, `AutonomousRiderJS` addresses the primary challenges of the Deliveroo.js environment. Sub-millisecond distance caching and synchronous concurrency controls ground the 2-slot intention queue, eliminating event-loop stalls and preserving high reactivity against rapid parcel decay. In parallel, dynamic spatial partitioning, soft coordination biases, and cooperative handoff choreography enable stable multi-agent coordination in constrained environments, resolving corridor deadlocks while allowing harvester and courier roles to emerge naturally without static assignment. Furthermore, the measured planning results give a concrete answer to *when* automated planning belongs in such an agent: because solver latency is largely invariant to problem size, a remote solver is affordable only where the world changes more slowly than the round trip, and the plan library (classical plans one fallback away, behind a circuit breaker) is what makes it safe to try anyway.